

Warm-ups croisés : Node.js, MINA, FreeBSD — A-EPI-006

Trois domaines, trois familles, aucune excuse

Ted “Tedsig42” Kouhouenou ted.sig42@tutamail.com

Summary: Module de raccord : couvrir les exemples du papier qui ne sont pas des CVE du noyau Linux. Node.js path.normalize, Apache MINA ObjectSerializationDecoder et les mappings FreeBSD/XNU montrent que les trois familles ne dépendent ni d'un langage, ni d'un éditeur, ni d'un noyau. Aucun exploit : uniquement des prédicats, des machines à états et des matrices de branches.

Prerequisites : M-EPI-001, M-EPI-002, bases de JavaScript et de Java

Version: 1.0

Contents

I Instructions **2**

II Instructions Communes **5**

II.1 Structure du Dépôt 5

II.2 Qu'est-ce qu'une Preuve de Concept Valide 5

II.3 Processus de Soumission 6

II.4 Règle des Fichiers Supplémentaires Interdits 7

II.5 Format du Rapport 7

II.6 Cibles Autorisées Uniquement 8

II.7 Citez Toutes Vos Sources 8

III Instructions sur l'IA **10**

III.1 Aperçu de la Politique 10

III.2 Utilisations Autorisées 10

III.3 Utilisations Interdites 11

III.4 Responsabilité Personnelle 11

III.5 Citation Obligatoire de l'IA 11

III.6 Évaluation par les Pairs et l'IA 12

III.7 Rappel Éthique 13

IV Avant-propos **14**

V Aperçu du module **15**

VI Exercise 00 **16**

VII Exercise 01 **18**

VIII Exercise 02 **20**

IX Exercise 03 **22**

X Exercise 04 **24**

XI Rendu et évaluation **27**

XII Sources générales **28**

Chapter I

Instructions

- Deux jours.
- Aucune cible réelle, aucun payload de désérialisation, aucun chemin traversé sur un vrai système de fichiers.
- Les tests Node.js se font avec `path.win32` sur des chaînes, sans appel à `fs`.
- Pour MINA, lisez les avis, les sources et les diffs ; ne construisez pas de flux sérialisé malveillant.
- Pour FreeBSD et XNU, travaillez à partir des avis publics et de la section 18.11 du papier. Distinguez ce qui est démontré de ce qui est seulement cartographié.

Format du rendu — en-tête obligatoire

Tout fichier rendu dans ce module doit commencer par l'en-tête ci-dessous, **adapté à la syntaxe de commentaire du langage** utilisé.

Modèle générique

```
/**
 * Module: <code module>
 * Exercise: <numéro exo>
 * File: <nom du fichier>
 * Author: <votre login JeuneCrack>
 *
 * Description: <une phrase, pas de roman>
 *
 * Return: <voir ci-dessous>
 */
```

Login

Votre identifiant **JeuneCrack** (ex: `tedsig42`). Pour un rendu en binôme : `login1/login2`. Ajoutez un handle Matrix après le login si vous le souhaitez (ex: `tedsig42 @tedsig42`).

Return — selon le type de fichier

Type de fichier	Valeur de Return
C (`.c`)	`return (0);` (succès), `return (-1);` (erreur), `return (valeur);` (résultat calculé)
Python (`.py`)	`0` (succès), `1` (erreur), `N/A` (analyse, script sans exit)
Shell (`.sh`)	`exit 0` (succès), `exit 1` (erreur)
Markdown (`.md`)	`N/A` — les fichiers d'analyse n'ont pas de valeur de retour
Exploit C/Python	`shell` (shell obtenu), `flag: <flag>` (flag capturé), `proof: <hash>` (preuve de concept)
Binaire (`.bin`)	`N/A` — pas d'en-tête pour les fichiers binaires

Exemples par langage

C (`.c`) :

```c
/**
 * Module: F-EXP-001
 * Exercise: ex02
 * File: exploit.c
 * Author: tedsig42
 *
 * Description: Buffer overflow with ret2libc
 *
 * Return: 0
 */
```

Python (`.py`) :

```python
# Module: F-WEB-001
# Exercise: ex05
# File: ssrf-exploit.py
# Author: tedsig42
#
# Description: Blind SSRF bypass via redirect
#
# Return: 0
```

Shell (`.sh`) :

```sh
# Module: F-SUP-001
# Exercise: ex03
# File: dependency-confusion.sh
# Author: tedsig42
#
# Description: Automated dependency confusion test
#
# Return: exit 0
```

Markdown (`.md`) — analyse, rapport :

```
<!--
Module: M-SRC-001
Exercise: ex04
File: audit-report.md
Author: tedsig42

Description: Code audit of vulnerable-httpd

Return: N/A
-->
```

Chapter II

Instructions Communes

II.1 Structure du Dépôt

Votre dépôt Git doit respecter la convention de nommage suivante pour tous les modules soumis. L'outil d'évaluation (`pendragon`) se base sur cette structure pour localiser et tester votre travail.

```
<login>/
+-- <MODULE-ID>-<slug>/         # ex. M-BLS-001-business-logic/
|   +-- <turnindir>/            # ex. ex00-pricing-manipulation/
|   |   +-- <file1>             # ex. poc-pricing.py
|   |   +-- <file2>             # ex. report.md
|   |   +-- ...
|   +-- <turnindir>/            # ex. ex01-race-conditions/
|   |   +-- ...
|   +-- ...
+-- ...
```

- **Répertoire du module** : le nom complet du répertoire tel qu'affiché dans votre curriculum (ex. `M-BLS-001-business-logic`).
- **Répertoire de rendu** : la valeur de `\turnindir` fournie dans chaque exercice (ex. `ex00-pricing-manipulation`).
- **Fichiers de l'exercice** : exactement les fichiers listés dans `\exfiles`. Ni fichiers supplémentaires, ni fichiers manquants.

II.2 Qu'est-ce qu'une Preuve de Concept Valide

Une PoC doit être :

1. **Reproductible**. N'importe qui avec le même environnement peut l'exécuter et observer la vulnérabilité. Les chemins codés en dur, les adresses IP absolues et les jetons expirés ne sont pas acceptables.

2. **Documentée**. Chaque étape est expliquée. L'évaluteur ne devrait pas avoir à deviner ce que fait le script.

3. **Ciblée**. L'exercice spécifie une cible autorisée (lab, maquette, binaire local ou plateforme désignée). Ne testez jamais les PoC sur des systèmes de production qui ne vous appartiennent pas.

4. **Minimale**. La PoC démontre *uniquement* la vulnérabilité en question. Les charges utiles non liées et le bruit sont pénalisés.

① Une PoC qui modifie silencieusement l'état du système, supprime des fichiers ou persiste des changements en dehors du périmètre de l'exercice sera rejetée. Si votre exploit nécessite un nettoyage, documentez les étapes de nettoyage dans votre rapport.

II.3 Processus de Soumission

1. Réalisez les livrables de l'exercice sur votre machine locale.

2. Committez et poussez vers votre dépôt Git assigné.

3. Le dépôt est gelé après la date limite.

4. Les pairs clonent votre dépôt et évaluent en utilisant :

   - `pendragon` pour l'exécution automatisée des tests (là où le module fournit des configurations mouli).
   - La grille manuelle pour la qualité du rapport, le style de code et la méthodologie.



Exécutez toujours `pendragon run <MODULE-ID> <ex_id> <path/to/student/repo>` avant votre commit final pour vérifier que votre soumission passe les vérifications automatisées.

Test avec pendragon

Si le module inclut une configuration mouli (fichiers JSON dans `resources/mouli/`), vous pouvez tester votre soumission localement :

```
pendragon run M-BLS-001 ex00 ~/student-repos/tedsig42
```

`pendragon` résout le chemin de l'exercice automatiquement en cherchant `<mod-fullname>/<ex-dirname>/` dans le répertoire étudiant donné.

Vérification de style avec caliban

Pour les modules qui nécessitent du code C, `caliban` vérifie la conformité du style :

```
caliban check ~/student-repos/tedsig42/M-BLS-001-business-logic
```

`caliban` applique la Norme 42 (taille des fonctions, nommage des variables, constructions interdites) adaptée aux contextes de recherche de vulnérabilités. Exécutez-le avant la soumission — un échec aux vérifications de style peut invalider l'exercice.

II.4 Règle des Fichiers Supplémentaires Interdits

- Votre dépôt doit contenir **uniquement** les fichiers listés dans `\exfiles` pour chaque exercice.
- Fichiers interdits : `.DS_Store`, `__pycache__`, `.venv/`, `node_modules/`, `.git/`, binaires compilés (sauf explicitement listés dans `exfiles`), fichiers d'échange d'éditeur, fichiers journaux et tout fichier non demandé.
- Utilisez `.gitignore` pour exclure automatiquement les fichiers indésirables.

II.5 Format du Rapport

Lorsqu'un rapport (`report.md`) est requis, il doit inclure :

1. **Classe de vulnérabilité** (identifiant CWE si applicable).
2. **Étapes de reproduction** — numérotées, prêtes à être copiées-collées.
3. **Charges utiles** — chaînes exactes, requêtes ou entrées utilisées.
4. **Impact** — ce qu'un attaquant peut accomplir.
5. **Mitigation** — comment le développeur devrait corriger.
6. **Sources** — toutes les ressources consultées (voir Section II.7).

En-tête de Rendu

Chaque fichier source doit inclure l'en-tête de rendu défini dans `../../../resources/latex/deliverable-header.tex`, adapté à la syntaxe de commentaire du langage utilisé.

II.6 Cibles Autorisées Uniquement

① Vous pouvez **uniquement** tester les cibles explicitement listées dans l'exercice (labs, serveurs locaux, maquettes, binaires désignés). Tester des systèmes de production réels, des applications tierces ou toute cible qui ne vous appartient pas ou pour laquelle vous n'avez pas d'autorisation écrite est interdit et peut entraîner une disqualification immédiate.

L'éthique du bug bounty s'applique tout au long de ce curriculum :

- Aucun test sur des programmes en production sans un accord de périmètre valide.
- Aucun déni de service sauf si l'exercice l'exige explicitement.
- Aucune exfiltration de données au-delà de ce qui est nécessaire pour démontrer la vulnérabilité.
- Si vous accédez accidentellement à des données réelles d'utilisateurs, arrêtez immédiatement et documentez ce qui s'est passé.

II.7 Citez Toutes Vos Sources

Toute ressource consultée pendant votre travail doit être citée. Cela inclut :

- Articles académiques et billets de blog.
- Rapports publics de bug bounty (HackerOne, Bugcrowd, etc.).
- Documentation officielle (RFC, pages de manuel, documentation fournisseur).
- **Outils d'IA** : nom du modèle, résumé de la question, date et méthode de vérification (voir le chapitre Instructions IA pour le format de citation exact).
- Extraits de code adaptés de projets open-source (avec attribution de licence si requis).

Format de citation

```
# R\'ef\'erences

- [1] PortSwigger, "Business Logic Flaws",
  https://portswigger.net/web-security/logic-flaws
  (consult\'e le 2025-03-15).
- [2] Rapport HackerOne #123456, "IDOR on /api/users",
  anonymis\'e, consult\'e via divulgation publique.
- [3] G\'en\'er\'e avec l'assistance de Claude 3.5 Sonnet
  le 2025-03-14. Question : "Write a Python script that...".
  V\'erifi\'e par : revue manuelle de chaque requ\^ete HTTP.
```

Attribution en ligne

Si un exercice vous demande explicitement d'utiliser une technique ou un outil spécifique, vous n'avez pas besoin de citer l'exercice lui-même comme source. Citez tout ce qui se trouve en dehors de l'énoncé de l'exercice.

Chapter III

Instructions sur l'IA

III.1 Aperçu de la Politique

Les outils de codage assistés par IA (ChatGPT, Claude, Copilot, Gemini, Cursor, Codeium, et similaires) sont **autorisés** tout au long de ce cursus. La recherche en sécurité est un domaine où la maîtrise de l'IA devient essentielle — apprendre à utiliser ces outils efficacement fait partie de votre formation.

Cependant, l'objectif est la **compréhension**, pas l'automatisation. Si vous ne pouvez pas expliquer chaque ligne de votre exploit sans l'aide de l'IA, vous n'avez pas terminé l'exercice.

① Utiliser l'IA pour contourner l'objectif pédagogique d'un exercice — par exemple, lui demander de rétro-ingéniérer un binaire pour ne pas avoir à apprendre le désassembleur, ou de générer une chaîne d'exploitation complète que vous ne comprenez pas — va à l'encontre du but du cursus et sera traité comme une fraude académique.

III.2 Utilisations Autorisées

- **Explication de concepts**. Demander à l'IA de clarifier une classe de vulnérabilité, un sous-système du noyau, ou un protocole réseau.
- **Échafaudage de code**. Générer du code de départ pour un harnais, un fuzzer, ou un squelette de PoC — puis le modifier et l'étendre vous-même.
- **Débogage**. Expliquer des messages d'erreur, la sortie d'un désassembleur, ou un comportement de crash inattendu.
- **Résumé de documentation**. Condenser une longue RFC ou un article technique en notes exploitables.
- **Traduction**. Traduire du contenu technique entre l'anglais et le français (les deux langues sont utilisées dans ce cursus).

- **Rédaction de rapports**. Demander de l'aide pour structurer un rapport de bug bounty — mais remplir le contenu technique vous-même.

III.3 Utilisations Interdites

- **Génération complète d'exploit**. Demander à l'IA d'écrire l'exploit complet d'un exercice, puis le soumettre sans le comprendre.
- **Contournement des contraintes de l'exercice**. Utiliser l'IA pour contourner une restriction que l'exercice impose explicitement (p. ex. « n'utilisez pas de scanners automatisés »).
- **Découverte automatisée de vulnérabilités**. Exécuter des outils de scan pilotés par IA qui remplacent l'analyse manuelle que vous êtes censé pratiquer.
- **Rédaction des grilles d'évaluation**. Vous n'êtes pas autorisé à utiliser l'IA pour générer des critères d'évaluation par les pairs ou des réponses.
- **Plagiat sans attribution**. Copier le résultat de l'IA sans le citer (voir Section III.5).

III.4 Responsabilité Personnelle

Lorsque vous demandez à l'IA de générer du code d'exploit, vous devenez responsable de chaque octet de ce résultat avant de l'utiliser contre une cible.

1. **Lisez le code généré** ligne par ligne.
2. **Identifiez toutes les hypothèses** faites par l'IA (architecture, version du noyau, drapeaux du compilateur, topologie réseau).
3. **Testez en isolation** avant d'exécuter contre une cible d'exercice.
4. **Comprenez les modes d'échec**. Si l'exploit généré plante ou se bloque, vous devez diagnostiquer et corriger le problème vous-même.



Si l'IA génère du code que vous ne comprenez pas, demandez-lui de l'expliquer. Si vous ne comprenez toujours pas après l'explication, le code n'est pas prêt à être utilisé. Passez à une approche plus simple que vous maîtrisez entièrement.

III.5 Citation Obligatoire de l'IA

Chaque fichier contenant du code généré ou assisté par l'IA doit inclure un commentaire de citation. Le format est :

```
# Generated with assistance from <tool> on <date>.
# Prompt: <one-line summary of the prompt>
# Verified by: <method used to verify the output>
```

Exemples

Python :

```
# Generated with assistance from Claude 3.5 Sonnet on 2025-03-14.
# Prompt: "Write a Python script that sends HTTP requests with
# modified price fields to /api/checkout/pay"
# Verified by: manual review of each request and response
```

C :

```
/* Generated with assistance from GitHub Copilot on 2025-03-14.
 * Prompt: "Implement a stack-based buffer overflow with
 * ret2libc for x86_64 Linux"
 * Verified by: compiled with -fno-stack-protector and
 * tested against local binary with ASLR disabled */
```

Markdown (rapport) :

```
<!--
Generated with assistance from ChatGPT-4 on 2025-03-14.
Prompt: "Structure a bug bounty report for IDOR with
sections: summary, steps, impact, mitigation"
Verified by: all technical content written manually,
AI only suggested structure
-->
```

Interactions multiples avec l'IA

Si vous avez utilisé plusieurs outils d'IA, ou itéré à travers plusieurs invites, citez chaque interaction significative. Un groupe d'invites connexes peut être résumé :

```
<!--
AI assistance used in this file:
- Claude 3.5 Sonnet (2025-03-14): harness skeleton generation
- ChatGPT-4 (2025-03-14): crash triage help
- Copilot (2025-03-15): inline autocomplete for loop logic
All generated code reviewed and modified manually.
-->
```

III.6 Évaluation par les Pairs et l'IA

Lors de l'évaluation par les pairs, l'évaluateur peut :

- Vous demander d'**expliquer tout code généré par l'IA** ligne par ligne.

- Vous demander de **reproduire le raisonnement** sans l'aide de l'IA.
- Demander à voir votre **historique d'invites** pour un exercice donné.
- Déduire des points si la citation de l'IA est manquante ou si vous ne pouvez pas expliquer le code que l'IA a généré.

① Présenter le code généré par l'IA comme votre propre travail (défaut de citation) est du plagiat. Si l'évaluateur soupçonne du plagiat, il peut saisir l'équipe pédagogique pour examen.

III.7 Rappel Éthique

- Les modèles d'IA peuvent produire du code incorrect, dangereux, ou illégal. Vous êtes **responsable** de vérifier chaque résultat avant exécution.
- Les modèles d'IA entraînés sur la recherche publique en sécurité peuvent reproduire textuellement du code provenant de projets sous licence GPL. Vérifiez les licences avant d'intégrer ces éléments dans votre travail.
- Ne saisissez pas de code sensible ou propriétaire dans des outils d'IA hébergés dans le cloud. Utilisez des modèles locaux (Llama, Mistral, DeepSeek) pour les travaux impliquant des cibles non publiques.
- L'éthique de la recherche en sécurité que vous apprenez ici s'applique à l'utilisation de l'IA : ne demandez pas à l'IA de générer des malwares, des rançongiciels, ou du code destiné à nuire à des systèmes en dehors du cadre de l'exercice.



La maîtrise des outils d'IA est une compétence recherchée dans la chasse aux bogues moderne. Utilisez-les sagement, citez-les honnêtement, et conservez toujours votre propre compréhension de chaque exploit que vous déployez.

Chapter IV

Avant-propos

Le papier ne parle pas seulement du noyau Linux. Il utilise trois *warm-ups* pour prouver que son modèle survit au changement de domaine :

Domaine	**Famille**	**Pourquoi ce cas compte**
Node.js	Predictive Oracle	Un prédicat de chemin croit modéliser Windows ; le système d'exploitation en sait plus que lui.
Apache MINA	Logical Engine	L'ordre entre initialisation et vérification est une transition d'état, pas un simple `if` manquant.
FreeBSD / XNU	Plusieurs	Le papier revendique une portée cross-kernel ; il faut vérifier ce qui est réellement documenté.



U

n *warm-up* n'est pas un exercice facile. C'est un cas réduit qui permet de voir la structure avant d'ajouter la complexité du noyau. Si vous savez expliquer Node.js et MINA, vous devez pouvoir expliquer nf_tables, eBPF et io_uring avec le même vocabulaire.

Chapter V

Aperçu du module

Ex	Titre	Difficulté	Points	
00	Node.js : le prédicat et le système	Moyen	4	
01	MINA : l'ordre des opérations	Moyen	4	
02	MINA : la matrice des branches	Difficile	4	
03	FreeBSD/XNU : ce que le papier ne prouve pas	Difficile	4	
04	Synthèse : tous les exemples du papier	Difficile	4	
	Total		**20**	

Chapter VI

Exercise 00

	Exercise 00
Node.js : le prédicat et le système	
Turn-in directory : `ex00/`	
Files to turn in : `node-paths.md path-model.js`	
Allowed functions : `Node.js, un éditeur de texte, documentation Windows`	

`path.normalize()` et `path.join()` analysent une chaîne avant que le système de fichiers ne l'utilise. C'est un Predictive Oracle : le prédicat décide qu'un chemin est sûr, puis le système d'exploitation lui donne un sens différent.

Le papier décrit trois générations :

CVE / cas	**Hypothèse du correctif**	**Angle mort**
CVE-2025-23084	Un chemin qui ne commence pas par une lettre de lecteur est relatif.	`C:..\sensitive` est relatif au répertoire courant du lecteur C:.
CVE-2025-27210	Filtrer les noms de lecteur suffit à bloquer les chemins Windows spéciaux.	`CON`, `PRN`, `AUX`, `NUL`, `COM1-9`, `LPT1-9` sont des périphériques, pas des répertoires.
UNC non assigné	Filtrer les noms de périphériques suffit.	Les chemins UNC `\\serveur\partage` restent valides et peuvent sortir du sous-arbre prévu.

Mission

Dans `node-paths.md` :

1. expliquez la différence entre `path.normalize`, `path.join` et la résolution finale par Windows ;

2. remplissez un tableau *entrée*, *sortie du prédicat*, *interprétation Windows*, *hypothèse violée* pour les trois cas ;

3. expliquez pourquoi chaque correctif est une extension du modèle, pas une fermeture de la classe ;

4. proposez un corpus de tests qui couvre lecteurs, noms réservés, UNC, chemins relatifs et séparateurs mélangés ;

5. classez la chaîne : *architectural scatter* ou *incomplete-fix variant* ? Justifiez.

Dans `path-model.js`, écrivez un petit script qui appelle uniquement `path.win32.normalize` et `path.win32.join` sur des chaînes fournies. Le script ne doit jamais appeler `fs`, `child_process` ou ouvrir un fichier.



N

e testez pas ces chemins sur un vrai disque Windows. Le but est de comparer le modèle du prédicat au modèle du système, pas de provoquer une traversée de répertoire.

Sources

- NVD, CVE-2025-23084 : https://nvd.nist.gov/vuln/detail/CVE-2025-23084
- NVD, CVE-2025-27210 : https://nvd.nist.gov/vuln/detail/CVE-2025-27210
- Node.js, security releases de janvier 2025 : https://nodejs.org/en/blog/vulnerability/january-2025-security-releases
- Node.js, security releases de juillet 2025 : https://nodejs.org/en/blog/vulnerability/july-2025-security-releases
- Node.js, documentation `path` : https://nodejs.org/api/path.html
- Microsoft, nommage des fichiers et chemins Windows : https://learn.microsoft.com/en-us/windows/win32/fileio/naming-a-file

Chapter VII

Exercise 01

	Exercise 01
MINA : l'ordre des opérations	
Turn-in directory : `ex01/`	
Files to turn in : `mina-fsm.md`	
Allowed functions : `un éditeur de texte, sources Java, avis Apache`	

Apache MINA utilise la désérialisation Java pour reconstruire des objets reçus du réseau. Le papier modélise ce chemin comme une machine à états :

```
allocate -> initialize -> resolve -> check
```

Le bug n'est pas « il manque un filtre ». Le bug est que la vérification arrive après une transition qui a déjà exécuté du code.

CVE	**Hypothèse**	**Transition asymétrique**
CVE-2024-52046	Aucun filtre n'est nécessaire si l'application fait confiance au flux.	Instanciation arbitraire avant tout contrôle.
CVE-2026-41409	Une allowlist vérifiée dans `AbstractIoBuffer.getObject()` protège l'instanciation.	Les initialiseurs statiques s'exécutent avant l'évaluation de la garde.
CVE-2026-42778	Le correctif de 2.0.x est complet pour toutes les branches.	2.1.x et 2.2.x reçoivent une seconde correction.

Mission

Dans `mina-fsm.md` :

1. dessinez la machine à états *allocate*, *initialize*, *resolve*, *check* ;
2. placez chaque CVE sur la transition qui est mal ordonnée ;

3. expliquez pourquoi une allowlist de classes ne suffit pas si elle est consultée trop tard ;

4. identifiez l'invariant qui devrait être vrai avant toute initialisation ;

5. proposez un test de régression conceptuel, sans payload.



N

e construisez pas de flux Java sérialisé malveillant. Lisez le code, les avis et les diffs ; décrivez la transition, pas la charge utile.

Sources

- NVD, CVE-2024-52046 : https://nvd.nist.gov/vuln/detail/CVE-2024-52046
- NVD, CVE-2026-41409 : https://nvd.nist.gov/vuln/detail/CVE-2026-41409
- NVD, CVE-2026-42778 : https://nvd.nist.gov/vuln/detail/CVE-2026-42778
- Apache MINA, annonce CVE-2024-52046 : https://lists.apache.org/thread/4wxktgjpggdbto15d515wdctohb0qmv8
- Apache MINA, annonce CVE-2026-41409 : https://lists.apache.org/thread/9ddvsq6c4l5bhwq8l14sob4f8qjvx5c9
- Apache MINA, annonce CVE-2026-42778 : https://lists.apache.org/thread/fhlx5k91hrkgyzh7yk1nghrn3k27gxy0
- Documentation Java sur les filtres de désérialisation : https://docs.oracle.com/en/java/javase/21/core/serialization-filtering1.html

Chapter VIII

Exercise 02

	Exercise 02
MINA : la matrice des branches	
Turn-in directory : `ex02/`	
Files to turn in : `branch-matrix.md`	
Allowed functions : `un éditeur de texte, avis Apache, historique des versions`	

Le cas MINA est un *Logical Engine* réparti sur plusieurs branches de maintenance. Une correction peut être complète sur une branche et incomplète sur une autre, sans nouveau chemin de code.

Mission

Remplissez `branch-matrix.md` avec les branches maintenues et les versions de correction connues :

Branche	**Version corrigée**	**CVE couverte**	**Preuve / incertitude**
2.0.x			
2.1.x			
2.2.x			

Puis répondez :

1. Pourquoi la version 2.0.28 ne prouve-t-elle pas que 2.1.11 est complète ?

2. Quelle hypothèse de release le correctif de 2.0.x encode-t-il ?

3. Comment vérifier qu'une branche a réellement reçu le déplacement de la garde, et pas seulement un changement de version ?

4. Pourquoi ce cas est-il un *incomplete-fix variant* et non un *architectural scatter* ?

5. Quelle question poser à un mainteneur avant de déclarer la classe fermée ?

Sources

- Phrack, *Security is failure: Every Patch Is a Guess*, section 13.2.
- Apache MINA, avis CVE-2026-41409 et CVE-2026-42778 (mêmes liens que l'exercice 01).
- NVD, CVE-2026-41409 et CVE-2026-42778.
- Apache MINA, page de téléchargement et d'archives : `https://mina.apache.org/mina-project/downloads.html`

Chapter IX

Exercise 03

	Exercise 03
FreeBSD/XNU : ce que le papier ne prouve pas	
Turn-in directory : `ex03/`	
Files to turn in : `cross-kernel.md`	
Allowed functions : `un éditeur de texte, NVD, avis FreeBSD, sources XNU`	

La section 18.11 du papier reconnaît une limite de portée : le framework a été développé sur Linux, puis validé sur Windows. Les mappings FreeBSD et XNU sont documentés dans l'archive, pas démontrés dans le corps de l'article.

Deux cas FreeBSD sont cités :

- CVE-2026-42511 : `dhclient` ;
- CVE-2026-4747 : NFS.

Le papier mentionne aussi XNU avec IOKit `IOUserClient` et Mach IPC comme surfaces de travail, sans en faire une étude de cas complète.

Mission

Dans `cross-kernel.md` :

1. pour chaque cas FreeBSD, identifiez le composant, la famille probable et l'hypothèse que le correctif encode ;
2. séparez explicitement *ce que le papier affirme*, *ce que l'avis public affirme* et *ce que vous inférez* ;
3. pour XNU, listez deux coutures d'audit (IOKit, Mach IPC) et la question de cycle de vie ou de frontière correspondante ;

4. expliquez pourquoi une classification cross-kernel peut être un *faux positif structurel* ;

5. concluez : le papier démontre-t-il la portée cross-kernel, ou propose-t-il une hypothèse de portée ?



N

'utilisez pas ces cas pour affirmer qu'une CVE est exploitable. Le papier lui-même rappelle que le framework produit des candidats, pas des vulnérabilités confirmées.

Sources

- Guide de lecture du papier : `00-guide/PAPER-GUIDE-paper/subject/PAPER-GUIDE-paper_fr.pdf`.
- NVD, CVE-2026-42511 : https://nvd.nist.gov/vuln/detail/CVE-2026-42511
- NVD, CVE-2026-4747 : https://nvd.nist.gov/vuln/detail/CVE-2026-4747
- FreeBSD, avis de sécurité : https://www.freebsd.org/security/advisories/
- Sources XNU : https://github.com/apple-oss-distributions/xnu
- Phrack, section 18.11, *What the Framework Does Not Find*.

Chapter X

Exercise 04

	Exercise 04
Une carte exhaustive de tous les exemples du papier	
Turn-in directory : `ex04/`	
Files to turn in : `paper-coverage.md`	
Allowed functions : `un éditeur de texte, le papier, les modules, NVD`	

Le papier contient plus d'exemples que les six CVE des études de cas principales. Cette carte sert à vérifier qu'aucun exemple n'a été oublié et à distinguer les preuves des extrapolations.

Mission

Remplissez `paper-coverage.md` pour les CVE suivantes. Indiquez la section du papier, la famille, le mode de mutation et le module EPIAG qui la couvre.

CVE	Section	Famille	Module / statut
CVE-2023-35001	6.2	Predictive Oracle	M-EPI-003, A-EPI-002
CVE-2024-1086	6.2	Predictive Oracle	M-EPI-003, A-EPI-002
CVE-2023-4244	6.3	Logical Engine	M-EPI-000, M-EPI-003
CVE-2024-1085	6.3	Logical Engine	M-EPI-000, M-EPI-003
CVE-2025-23084	13.1	Predictive Oracle	A-EPI-006
CVE-2025-27210	13.1	Predictive Oracle	A-EPI-006
UNC non assigné	13.1	Predictive Oracle	A-EPI-006
CVE-2024-52046	13.2	Logical Engine	A-EPI-006
CVE-2026-41409	13.2	Logical Engine	A-EPI-006
CVE-2026-42778	13.2	Logical Engine	A-EPI-006
CVE-2020-27194	14.2	Predictive Oracle	M-EPI-004, A-EPI-003
CVE-2021-3490	14.3	Predictive Oracle	M-EPI-004, A-EPI-003
CVE-2022-23222	14.4	Predictive Oracle	M-EPI-004, A-EPI-003
CVE-2023-2163	14.5	Predictive Oracle	M-EPI-004, A-EPI-003
CVE-2021-20226	15.2	Concurrent Proxy	M-EPI-004, A-EPI-003
CVE-2021-3491	15.2	Concurrent Proxy	M-EPI-004, A-EPI-003
CVE-2023-2598	15.3	Concurrent Proxy	M-EPI-004, A-EPI-003
CVE-2024-0582	15.3	Concurrent Proxy	M-EPI-004, A-EPI-003
CVE-2025-33073	16.2	Logical Engine	A-EPI-004
CVE-2025-58726	16.2	Logical Engine	A-EPI-004
CVE-2026-24294	16.2	Logical Engine	A-EPI-004
CVE-2026-26128	16.2	Logical Engine	A-EPI-004
CVE-2026-26119	16.2	Logical Engine	A-EPI-004
CVE-2026-42511	18.11	Logical Engine ?	A-EPI-006
CVE-2026-4747	18.11	Concurrent Proxy ?	A-EPI-006

Puis ajoutez trois paragraphes :

1. **Démontré** : quels cas ont une analyse détaillée dans le corps du papier ?

2. **Observé** : quels cas viennent du corpus de 62 CVE ou de l'archive, sans analyse complète ?

3. **Extrapolé** : quels cas servent à revendiquer une portée cross-kernel sans preuve détaillée ?

Enfin, répondez à la question de soutenance : « Si vous deviez défendre ce papier, quelle serait votre affirmation la plus forte et votre concession la plus honnête ? »

Sources

- Phrack, *Security is failure: Every Patch Is a Guess*, sections 6, 13, 14, 15, 16 et 18.11.
- Dépôt compagnon : `validation/dataset.csv` et `validation/methodology.md`.
- NVD : `https://nvd.nist.gov/`
- Les modules EPIAG cités dans le tableau.

Chapter XI

Rendu et évaluation

```
A-EPI-006-warmups-cross-domain/
|-- ex00/
|   |-- node-paths.md
|   +-- path-model.js
|-- ex01/mina-fsm.md
|-- ex02/branch-matrix.md
|-- ex03/cross-kernel.md
+-- ex04/paper-coverage.md
```

Critère	**Attendu**	**Points**
Node.js	Prédicat, sémantique Windows et trois angles morts	4
MINA	Ordre des transitions et invariant explicites	4
Branches	Matrice de versions et preuves sourcées	4
Cross-kernel	Distinction démontré / observé / extrapolé	4
Synthèse	Tous les exemples couverts, concession honnête	4

Chapter XII

Sources générales

- Phrack, *Security is failure: Every Patch Is a Guess*, sections 13, 18.9 et 18.11.
- Node.js, avis de sécurité 2025 et documentation `path`.
- Apache MINA, avis CVE-2024-52046, CVE-2026-41409 et CVE-2026-42778.
- FreeBSD, avis de sécurité et NVD pour CVE-2026-42511 et CVE-2026-4747.
- Dépôt compagnon : `cross-kernel/`, `validation/` et `rules/`.